
MetaSignal

Streaming Extension PRD vNext

Hybrid Batch + Streaming Metric Intelligence Platform

“Streaming gives speed. Batch gives authority. Reconciliation gives trust.”

PARKED vNext EXTENSION · NOT PART OF METASIGNAL v1

NON-PRODUCTION · PRODUCTION-SIMULATED · BATCH REMAINS AUTHORITATIVE

Sidharth Kriplani · April 2026 · Solo-built, non-production, production-simulated

Companion: MetaSignal PRD v2.0 · This extension does not replace the batch experiment system.

A. Executive Summary

This document is a parked vNext extension to MetaSignal PRD v2. It defines how MetaSignal can evolve from a batch-first experimentation platform to a hybrid batch plus streaming metric intelligence platform without compromising statistical validity, without replacing batch authority, and without building unnecessary infrastructure.

What this extension is. A narrowly scoped streaming monitoring layer that adds provisional early-warning detection — SRM, instrumentation gaps, traffic drift, event quality — in the first hours of an experiment.

Why it exists. Batch is authoritative but slow. A broken assignment or a platform-wide instrumentation failure may not surface until the next daily run. Streaming closes that operational visibility gap.

Why it is parked vNext. MetaSignal v1 does not need streaming to be credible. CUPED, A/A validation, guardrail-first decisioning, right-censoring, and post-launch validation are the core. Streaming is additive depth after the core is solid.

How it extends without replacing. The batch pipeline, Postgres registry, DuckDB/dbt compute, APScheduler, and decision logs remain unchanged. Streaming adds a parallel event monitor with its own artifacts, reconciled against batch truth at daily readout.

One-line positioning: MetaSignal's streaming monitoring layer provides provisional SRM early warning and instrumentation health detection — streaming gives speed, batch gives authority, reconciliation gives trust.

B. Why MetaSignal Needs This Extension

MetaSignal v1 is a batch-first system. The batch pipeline computes correct, variance-reduced, guardrail-gated metrics on a daily schedule. That is correct by design. Batch is the authoritative layer. Nothing in this extension challenges that.

The gap is operational visibility in the first few hours. Batch detects a broken assignment mechanism — or a complete platform instrumentation failure — only after the daily run completes. For a serious experiment, that is a meaningful delay.

Operational gap	Current batch behaviour	What streaming adds
Broken assignment (SRM)	Detected in next daily run	<i>Provisional alert within 30 minutes</i>
Platform instrumentation drop	Visible next morning	<i>Event flow gap flagged within minutes</i>
Traffic allocation drift	Captured in daily SRM test	<i>Rolling allocation snapshot every 5 min</i>
Consumer / processing lag	Not monitored	<i>consumer_lag_report.json per window</i>
Malformed or duplicate events	Counted at batch load	<i>DLQ quarantine + duplicate report in realtime</i>

Important: MetaSignal core is still the priority. Streaming is additive depth. MetaSignal v1 is fully credible without this extension.

C. What Streaming Should Do

The streaming layer has eight narrow responsibilities. Each is well-defined, artifact-producing, and provisional.

SRM early warning.

Detects unexpected treatment/control split in rolling windows. Writes srm_streaming_alerts. Triggers analyst investigation, never ship/ramp decisions.

Traffic allocation monitoring.

Tracks variant share every 5 minutes against expected allocation. Writes `traffic_allocation_snapshots`. Flags sustained drift for review.

Instrumentation health / event flow gap detection.

Establishes per-platform/segment event flow baseline. Detects hard drops (e.g. iOS events disappear). Writes `instrumentation_alerts`.

Consumer lag / processing lag detection.

Monitors gap between `event_time` and `processing_time`. Writes `consumer_lag_report.json` per window. Triggers lag-aware alert suppression.

Late-event detection.

Events arriving beyond allowed lateness window are counted and routed to `late_event_report.json`. Batch handles late events authoritatively.

Malformed-event quarantine / DLQ.

Events failing schema validation are written to `dlq_events.parquet` with `error_code` and original payload. Never included in provisional metrics.

Provisional anomaly alerts.

Rolling z-score and EWMA on metric snapshots. Every alert is labelled `provisional_realtime_alert`. An anomaly is an investigation trigger, not a decision.

Stream-batch reconciliation.

After each daily batch run, streaming provisional values are compared to authoritative batch values. Deltas are documented. Repeated mismatch flags streaming logic as unreliable.

D. What Streaming Must Not Do

Rule: Streaming alerts may trigger investigation or pause-for-review, but never final experiment decisions.

- Produce final experiment readouts or daily metric values
- Make ship, ramp, or rollback decisions
- Replace CUPED variance reduction
- Replace batch guardrail gates
- Evaluate delayed or right-censored metrics as final
- Connect directly to the experiment decision engine
- Encourage peeking or generate false confidence in early metric directions
- Become a Kafka/Flink infrastructure project
- Suppress or override batch truth when the two disagree
- Be presented as a real-time decisioning layer

The peeking problem. Streaming provisional metrics move continuously. An analyst watching a rolling CVR delta is exposed to many interim looks. Streaming metrics must be framed as health signals — is the experiment instrumented correctly? — not as early lifts. The display layer must make the provisional label visible and unavoidable.

E. Batch vs. Streaming Responsibility Split

Layer	Responsibilities
<i>Streaming</i>	Provisional rolling metric snapshots · SRM early warning · Event flow monitoring · Traffic allocation snapshots · Consumer lag · Late event detection · Bad-event DLQ · Provisional anomaly alerts
Batch	Official daily metric computation · CUPED variance reduction · A/A validation · Final SRM confirmation · Guardrail gates · Right-censoring and delayed maturity · Post-launch validation · Decision logs · Official experiment readout
	<i>Comparing provisional streaming values to authoritative batch values · Delta computation and threshold check · Calibrating streaming alert thresholds · Confirming or suppressing provisional alerts · Proving streaming layer trustworthiness over time</i>

F. Architecture

The extension uses Python asyncio for the portfolio implementation. No Kafka, Redpanda, or Flink is required. The EventSource interface keeps application logic broker-agnostic: AsyncSimulatorEventSource is the initial implementation; RedpandaKafkaEventSource is an optional future drop-in.

Component	Role
<code>stream_event_generator.py</code>	Async event simulator. Emits events at configurable rate with injection hooks.
EventSource (interface)	Broker-agnostic abstraction. Enables future Redpanda/Kafka swap without logic changes.
AsyncEventQueue	In-memory asyncio queue (or append-only Parquet log for replay). Buffers emitted events.
StreamEventConsumer	Consumes events from queue. Routes to validators and monitors.
StreamValidator	Schema validation. Routes invalid events to DeadLetterQueueWriter.
SRMDetector	Sliding window split monitor. Computes deviation_pp vs expected. Writes srm_streaming_alerts.
TrafficAllocationMonitor	5-min variant share tracking. Writes traffic_allocation_snapshots.
InstrumentationHealthMonitor	Per-platform event flow baseline. Hard drop detection. Writes instrumentation_alerts.
ConsumerLagMonitor	Tracks event_time vs processing_time lag (p50/p95/p99). Writes consumer_lag_report.json.
LateEventDetector	Events beyond allowed_lateness_minutes routed here. Writes late_event_report.json.
DeadLetterQueueWriter	Malformed/schema-failed events written to dlq_events.parquet with error codes.
RealtimeMetricSnapshotter	5-min rolling metric aggregates. Every row carries provisional_label=True.
ProvisionalAnomalyDetector	Rolling z-score + EWMA on snapshots. Lag-aware suppression. Writes provisional_anomaly_alerts.
StreamBatchReconciler	Post-batch: compares provisional values to authoritative values. Writes stream_batch_reconciliation_report.json.
<code>show_streaming_demo.py</code>	Terminal demo runner. Injects scenarios, shows alerts, runs reconciliation.

G. Event Schema

Every event flowing through the streaming layer conforms to this schema. Schema validation is enforced at StreamValidator before any downstream processing.

Field	Type	Notes
event_id	UUID	Globally unique. Used for duplicate detection.
event_time	ISO-8601	When the event occurred on the client. Used for windowing.
processing_time	ISO-8601	When the event arrived at the consumer. Lag = processing_time - event_time.
user_id	STRING	Randomisation unit. Required for SRM computation.
session_id	STRING	Session identifier. Useful for session-grain metrics.
experiment_id	STRING	Required. Events without experiment_id go to DLQ.
variant	STRING	control or treatment. Required. Events with invalid variant go to DLQ.
metric_event_type	STRING	e.g. page_view, add_to_cart, purchase, return_initiated.
platform	STRING	ios, android, web. Used for per-platform instrumentation health.
country	STRING	ISO country code. Used for segment flow monitoring.
page_type	STRING	e.g. homepage, pdp, cart, checkout.
value	FLOAT	Metric value. Null for binary events.
schema_version	STRING	e.g. v1.2. Schema drift between versions triggers DLQ routing.
ingestion_source	STRING	Which upstream emitter produced this event.
is_late_event	BOOL	Set by LateEventDetector if event_time is before watermark.
is_duplicate	BOOL	Set if event_id seen before in deduplication window.
validation_status	STRING	valid / invalid_schema / invalid_variant / duplicate / late.

event_time vs processing_time. event_time is when the event happened on the user's device or server. processing_time is when the consumer received it. The difference is lag. Late events have event_times that fall before the current watermark. Both timestamps must be present.

Why schema_version matters. When the instrumentation team deploys a new event schema, old and new events co-exist in the stream. Schema drift detection prevents metric computation on incompatible event formats.

H. New Tables and Artifacts

streaming_metric_snapshots

Purpose: 5-min rolling metric aggregates per experiment/variant.

Key fields: *metric_id, experiment_id, variant, window_start, window_end, event_count, metric_value, provisional_label=True, window_complete*

Staff signal: Shows provisional metric direction. Never used for decisions. interview: provisional_label is always True.

srm_streaming_alerts

Purpose: Rolling-window SRM deviation alerts.

Key fields: [experiment_id](#), [window_start](#), [window_end](#), [expected_control_pct](#), [observed_control_pct](#), [deviation_pp](#), [sample_size](#), [alert_level](#), [provisional_status](#), [batch_confirmation_status](#)

Staff signal: Proves SRM detection is active and provisional. [batch_confirmation_status](#) resolves the alert after daily run.

traffic_allocation_snapshots

Purpose: 5-min variant share tracking.

Key fields: [experiment_id](#), [variant](#), [window_start](#), [event_count](#), [share_pct](#), [expected_pct](#), [delta_pp](#)

Staff signal: Proves traffic allocation monitoring is operational.

instrumentation_alerts

Purpose: Per-platform/segment event flow gap flags.

Key fields: [experiment_id](#), [platform](#), [segment](#), [window_start](#), [baseline_event_count](#), [observed_event_count](#), [drop_pct](#), [alert_level](#), [suspected_cause](#)

Staff signal: Proves instrumentation health monitoring works. iOS drop scenario fires this artifact.

consumer_lag_report.json

Purpose: Per-window lag distribution (p50/p95/p99).

Key fields: [window_start](#), [p50_lag_seconds](#), [p95_lag_seconds](#), [p99_lag_seconds](#), [max_lag_seconds](#), [alert_level](#)

Staff signal: Proves the system knows when processing is delayed and can suppress lag-affected anomaly alerts.

late_event_report.json

Purpose: Events arriving after their allowed lateness window.

Key fields: [window_start](#), [late_event_count](#), [total_event_count](#), [late_pct](#), [affected_experiments](#), [handling:](#) [batch_or_drop](#)

Staff signal: Proves late-event handling is designed. Batch handles late events authoritatively.

duplicate_event_report.json

Purpose: Duplicate event_id detections per window.

Key fields: [window_start](#), [duplicate_count](#), [total_event_count](#), [duplicate_pct](#), [dedup_method](#)

Staff signal: Proves deduplication is active and measurable.

dlq_events.parquet

Purpose: All schema-invalid or unroutable events.

Key fields: [event_id](#), [event_time](#), [processing_time](#), [error_code](#), [error_detail](#), [original_payload](#) (JSON)

Staff signal: Proves bad events are quarantined, not silently dropped or miscomputed.

stream_quality_checks

Purpose: Per-window health gate results.

Key fields: [window_start](#), [check_name](#), [status](#) (pass/warn/fail), [value](#), [threshold](#)

Staff signal: Proves the streaming layer self-monitors its own quality before surfacing metrics.

provisional_anomaly_alerts

Purpose: Rolling z-score / EWMA anomaly detections.

Key fields: *metric_id, experiment_id, window_start, z_score, ewma_delta, alert_type, provisional_label=True, lag_suppressed (bool)*

Staff signal: All alerts carry provisional_realtime_alert label. Batch later confirms or suppresses.

stream_batch_reconciliation_report.json

Purpose: Daily comparison of streaming vs batch truth.

Key fields: *metric_id, date, streaming_value, batch_value, absolute_delta, relative_delta, late_event_count, duplicate_event_count, dlq_event_count, reconciliation_status, suspected_cause, calibration_note*

Staff signal: The most important artifact. Proves streaming is calibrated against batch. Repeated mismatch flags streaming as unreliable.

streaming_health_report.json

Purpose: End-of-window system health summary.

Key fields: *window_start, total_events, valid_events, dlq_events, late_events, duplicates, srm_alerts_active, instrumentation_alerts_active, consumer_lag_p95, overall_health_status*

Staff signal: Proves the streaming layer is self-monitoring. Single-glance system health.

replay_run_log.json

Purpose: Audit trail for replay / backfill runs.

Key fields: *replay_id, triggered_by, window_range_replayed, events_reprocessed, artifacts_updated, batch_truth_unchanged (bool), completed_at*

Staff signal: Proves replay repairs provisional artifacts without touching batch authority.

I. Windowing and Late-Event Logic

Streaming metrics are computed over time windows, not instantaneously. Window design determines how fresh, how complete, and how trustworthy provisional metrics are.

Window type	Use in this extension	Note
Tumbling (5-min)	Metric snapshots, traffic allocation	Non-overlapping. Each window is independent.
Sliding (30-min)	SRM detection, consumer lag	Overlapping. More stable split estimates.
Daily (batch)	Reconciliation anchor	Authoritative. Batch computes the official value.

Allowed lateness and watermarks

A watermark is the system's current estimate of the oldest event_time it should still accept into a window. Events with event_time before the watermark are late. allowed_lateness_minutes is configurable (recommended default: 15 minutes). Events arriving within the lateness window are included in the provisional metric. Events arriving beyond the lateness window are routed to late_event_report.json and handled by batch. This means provisional metrics can change as late events arrive within the allowed window — another reason they are never final.

Why final numbers belong to batch

Batch processes all events regardless of lateness. Batch applies CUPED, grain logic, denominator rules, and right-censoring. Streaming does none of this. A streaming CVR is an unweighted count ratio over a time window. A

batch CVR is a CUPED-adjusted per-user metric with correct denominator governance. They measure different things.

J. SRM Early Warning Design

SRM (Sample Ratio Mismatch) is the most critical early-warning use case. A broken assignment mechanism can invalidate an entire experiment. Detecting it in the first 30 minutes rather than the next morning is operationally significant.

- **Expected split.** Loaded from experiment config at SRMDetector startup. Default 50/50.
- **Observed split.** Computed from distinct user_id counts per variant in the sliding window.
- **Minimum sample threshold.** No alert is fired until the window has at least N distinct users (default: 500 per variant). Below threshold: provisional_status = insufficient_sample.
- **Deviation threshold.** $|\text{observed_control_pct} - \text{expected_control_pct}| > 2.0\text{pp}$ triggers a warning. $> 5.0\text{pp}$ triggers an investigation alert.
- **Alert severity levels.** none / warn (2-5pp) / investigate (>5pp).
- **False positive control.** Minimum sample gate, minimum window duration (15 minutes), and lag-aware suppression (if consumer_lag_p95 > 300 seconds, SRM alerts are suppressed) reduce false positives from processing lag artefacts.
- **Provisional status.** Every srm_streaming_alert has provisional_status = pending_batch_confirmation until the daily batch run completes the official SRM chi-square test.
- **Batch confirmation.** After daily run, batch_confirmation_status is updated to confirmed / suppressed / inconclusive.

Field	Description
experiment_id	Experiment being monitored.
window_start / window_end	30-minute sliding window boundaries.
expected_control_pct	From experiment config. e.g. 0.50.
observed_control_pct	Computed from distinct user_ids in window.
deviation_pp	$ \text{observed} - \text{expected} $ in percentage points.
sample_size	Total distinct users in window.
alert_level	none / warn / investigate.
provisional_status	insufficient_sample / active / pending_batch_confirmation.
batch_confirmation_status	pending / confirmed / suppressed / inconclusive.

K. Instrumentation Health Design

Instrumentation failures are often invisible until they affect metrics. A platform-wide event drop looks like a conversion rate improvement until you notice iOS sent no events for two hours. InstrumentationHealthMonitor catches this early.

- **Event flow baseline.** Computed from the first N complete windows of the experiment per platform/segment. Stored as expected_event_count_per_window.
- **Rolling drop detection.** If $\text{observed_event_count} < 30\%$ of baseline: alert_level = investigate. If zero events for 2 consecutive windows: alert_level = critical.

- **Missing segment detection.** If a platform or country that was active in prior windows sends zero events: `missing_segment_alert` fires.
- **Schema drift detection.** If `schema_version` distribution shifts materially within a window (e.g. v1.1 disappears): `schema_drift_alert` fires.
- **Malformed event detection.** Events failing `StreamValidator` are counted per window. If `malformed_rate > 5%`: `instrumentation_alert` with `alert_type = high_malformed_rate`.
- **Platform outage scenario.** iOS purchase events drop to zero for 90 minutes. `InstrumentationHealthMonitor` fires `instrumentation_alert` (`platform=ios`, `alert_level=critical`). Analyst investigates before batch readout. Batch later confirms and caveats the experiment readout.

L. Provisional Anomaly Detection

Streaming anomaly detection is intentionally simple. Complex statistical models on provisional data add false confidence. The goal is: flag anomalies worth investigating, not quantify their significance.

- **Rolling z-score.** Computed over the last N windows. $Z > 3$ triggers warn. $Z > 5$ triggers investigate.
- **EWMA (Exponentially Weighted Moving Average).** Smooth short-term noise. A sustained EWMA delta > 2 standard deviations from baseline triggers an alert.
- **Sudden drop detector.** If `metric_value` drops $> 40\%$ vs the prior window: `sudden_drop` alert regardless of z-score.
- **Minimum event volume threshold.** No anomaly alert fires if `event_count < minimum_sample`. Prevents noise from sparse windows.
- **Lag-aware suppression.** If `consumer_lag_p95 > lag_suppression_threshold_seconds`, anomaly alerts are suppressed (`lag_suppressed=True`). A processing delay looks like a metric drop.

Every provisional anomaly alert carries: `provisional_label = True`, `alert_category = provisional_realtime_alert`. A streaming anomaly is not business truth. It is an investigation trigger. The batch readout will confirm or suppress it.

M. Stream-Batch Reconciliation

Reconciliation is the most important component of the streaming extension. Without reconciliation, streaming metrics are unverified provisional numbers. With reconciliation, the streaming layer proves it is calibrated against authoritative batch truth — and repeated mismatch proves it is not.

Why reconciliation is more important than streaming itself

Any system can emit provisional metrics. The hard question is: do those provisional metrics agree with the authoritative batch truth? If they systematically diverge, the streaming layer is producing noise, not signal. Reconciliation is the evidence layer that answers that question.

Field	Description
<code>metric_id</code>	Which metric was compared.
<code>date</code>	The batch readout date this reconciliation covers.
<code>streaming_value</code>	End-of-day aggregation of provisional snapshots.
<code>batch_value</code>	Official CUPED-adjusted batch value.
<code>absolute_delta</code>	<code>streaming_value - batch_value</code> .

Field	Description
<code>relative_delta</code>	<code>absolute_delta</code> / <code>batch_value</code> . Core calibration metric.
<code>late_event_count</code>	Events that arrived after allowed lateness window.
<code>duplicate_event_count</code>	Deduplicated events not counted in streaming.
<code>dlq_event_count</code>	Events quarantined to DLQ (not in either metric).
<code>missing_event_count</code>	Events expected but not received (estimated from baseline).
<code>reconciliation_status</code>	match / minor_delta / investigate.
<code>suspected_cause</code>	late_events / duplicate_events / dlq_events / cuped_adjustment / unknown.
<code>threshold_used</code>	The <code>relative_delta</code> threshold that defines match vs minor_delta.
<code>calibration_note</code>	Free text. E.g. 'CUPED reduced variance by 18%. Expected delta.'

- **Why deltas are expected.** Streaming does not apply CUPED. CUPED reduces variance but also shifts the point estimate. A systematic `relative_delta` of 5-15% between streaming and batch is expected purely from CUPED. Reconciliation documents this as a calibration note, not a failure.
- **How repeated mismatch shows unreliability.** If `relative_delta` > threshold for 3+ consecutive days with no obvious cause (no late events, no DLQ volume), the streaming metric computation logic has a bug. `reconciliation_status` = investigate accumulates into a `streaming_quality_flag`.
- **How reconciliation calibrates alert thresholds.** If streaming consistently underestimates by 8% due to late events, alert thresholds should be adjusted downward by 8% to reduce false positives.

N. Failure and Recovery Scenarios

1. SRM injection — treatment receives 54% instead of 50%

Trigger: `stream_event_generator` injects imbalanced variant assignment (54/46 split).

Streaming behaviour: SRMDetector fires `srm_streaming_alerts`: `deviation_pp=4.0`, `alert_level=warn` after 500 users. Escalates to investigate at `deviation_pp=5.0`.

Batch reconciliation: Batch SRM chi-square test confirms split imbalance. `batch_confirmation_status=confirmed`. Experiment readout caveated.

Recovery: Analyst pauses experiment for assignment investigation. No automatic action.

2. iOS event flow gap — zero events for 90 minutes

Trigger: `stream_event_generator` stops emitting iOS events for 90-minute window.

Streaming behaviour: InstrumentationHealthMonitor detects drop to zero within 2 windows. `alert_level=critical`, `platform=ios`.

Batch reconciliation: Batch confirms iOS data gap. Experiment readout for iOS segment caveated or excluded.

Recovery: Analyst investigates iOS SDK instrumentation. Batch handles gap with right-censoring.

3. Late-arriving events — 20% arrive after window close

Trigger: 20% of purchase events have event_time 20 minutes before processing_time. Many fall past watermark.

Streaming behaviour: LateEventDetector routes events to late_event_report.json. Provisional metric undercounts by ~20%.

Batch reconciliation: Batch receives all events regardless. reconciliation_report shows relative_delta ~20%. Suspected_cause = late_events.

Recovery: Streaming alert thresholds recalibrated upward to account for systematic undercounting.

4. Duplicate events — same event_id emitted twice

Trigger: stream_event_generator emits event_id duplicates for 5% of purchase events.

Streaming behaviour: StreamValidator detects is_duplicate=True. Events counted in duplicate_event_report. Not included in metric snapshots.

Batch reconciliation: Batch deduplicates on its own grain. Reconciliation shows minor delta from dedup methodology difference.

Recovery: DLQ writer logs duplicates. Duplicate rate tracked as quality metric.

5. Malformed event — missing experiment_id or variant

Trigger: Events emitted without experiment_id or with variant=None.

Streaming behaviour: StreamValidator routes to DLQ: error_code=MISSING_REQUIRED_FIELD. dlq_events.parquet populated.

Batch reconciliation: Batch may also reject these events. DLQ count documented in reconciliation_report.

Recovery: Instrumentation team fixes upstream event emission. DLQ count tracked as instrumentation KPI.

6. Consumer lag spike — processing delayed by 20 minutes

Trigger: processing_time lags event_time by 20+ minutes across all events.

Streaming behaviour: ConsumerLagMonitor: p95_lag_seconds > lag_suppression_threshold. ProvisionalAnomalyDetector: lag_suppressed=True on all anomaly alerts.

Batch reconciliation: Batch is unaffected (batch processes all events post-window). Reconciliation notes lag as suspected_cause for delta.

Recovery: Anomaly alerts resume when lag returns to normal. No false positives during lag window.

7. Provisional anomaly false alarm — streaming fires, batch suppresses

Trigger: EWMA delta > 2 sigma on CVR for 3 consecutive windows due to sparse early traffic.

Streaming behaviour: ProvisionalAnomalyDetector fires: provisional_realtime_alert. Analyst notified for investigation.

Batch reconciliation: Batch daily run: CUPED-adjusted CVR is stable. Reconciliation: batch_value stable, streaming_value affected by sparse window. batch_confirmation_status=suppressed.

Recovery: False positive rate tracked in streaming_health_report. Alert thresholds adjusted.

8. Stream-batch mismatch — streaming says CVR down, batch says stable

Trigger: Systematic undercount in streaming due to late events + CUPED adjustment gap.

Streaming behaviour: Provisional CVR snapshot shows -8% delta. provisional_anomaly_alert fires.

Batch reconciliation: Batch shows stable CVR (CUPED-adjusted). reconciliation_report: relative_delta=8%, suspected_cause=cuped_adjustment+late_events, reconciliation_status=minor_delta.

Recovery: calibration_note added: expected delta from CUPED. Alert thresholds recalibrated.

9. DLQ poison event — bad schema crashes validator

Trigger: Event with unexpected field type (e.g. value='NULL_STRING' instead of FLOAT) reaches StreamValidator.

Streaming behaviour: StreamValidator catches TypeError, routes to DLQ with error_code=SCHEMA_TYPE_MISMATCH. Consumer continues.

Batch reconciliation: Batch schema validation catches same class of events. DLQ count reconciled.

Recovery: Poison events do not crash the consumer. DLQ acts as quarantine, not a halt trigger.

10. Replay recovery — consumer failure misses 2 hours of windows

Trigger: AsyncEventConsumer goes offline for 2 hours. Events buffered in AsyncEventQueue.

Streaming behaviour: Streaming metrics have 2-hour gap. streaming_health_report flags missing windows.

Batch reconciliation: Batch unaffected. After consumer recovers, replay_run_log.json documents reprocessed windows.

Recovery: Provisional artifacts repaired by replay. Batch truth unchanged. replay_run_log shows batch_truth_unchanged=True.

O. Replay and Backfill Design

Streaming consumers can fail. When the consumer recovers, windows that were missed need to be reprocessed from the buffered event log. Replay repairs provisional streaming artifacts without touching batch authority.

- **When replay is triggered.** Consumer restart after downtime, after a bug fix that changes window computation logic, or after schema migration.
- **What replay_run_log captures.** replay_id, triggered_by (manual/auto), window_range_replayed, events_reprocessed, artifacts_updated, batch_truth_unchanged=True, completed_at.
- **Why replay does not rewrite batch truth.** Batch operates independently on the full event history. Replay only updates streaming_metric_snapshots, srm_streaming_alerts, and related provisional artifacts.
- **How replay repairs reconciliation.** After replay, StreamBatchReconciler re-runs for the affected date range. If the repaired streaming values now agree with batch, reconciliation_status updates from investigate to match.

P. Integration With Existing MetaSignal

The streaming extension is additive. The existing MetaSignal system is unchanged.

Component	Status in vNext
Synthetic event generator (batch)	Unchanged
DuckDB + dbt + Parquet batch compute	Unchanged
Postgres metric registry / audit	Unchanged

Component	Status in vNext
APScheduler daily jobs	<i>Unchanged</i>
CUPED / A/A validation / guardrails	<i>Unchanged. Authoritative.</i>
Right-censoring / post-launch validation	<i>Unchanged</i>
Decision logs / show_demo_report.py	<i>Unchanged</i>
stream_event_generator.py	NEW — async streaming simulator
Streaming monitors + artifact writers	NEW — all components in section F
StreamBatchReconciler	NEW — daily post-batch reconciliation job
show_streaming_demo.py	NEW — standalone streaming demo runner

MetaSignal v1 works without streaming. Streaming is optional depth. The two systems share the experiment registry and metric definitions, but operate independent compute pipelines.

Q. Future PulseRank Integration

This section describes a future integration only. It is not a dependency for either MetaSignal streaming or PulseRank v1.

PulseRank produces recommendation and ranking decisions. A natural future state is for PulseRank to emit structured events that MetaSignal streaming can monitor for SRM, traffic allocation, and instrumentation health across ranking experiments.

recommendation_impression, recommendation_click, purchase, return_initiated
 ranking_variant_id, experiment_id, user_id, item_id, position
 recommendation_latency, exploration_flag

With these events, MetaSignal streaming could monitor: ranking experiment SRM, exploration traffic allocation, recommendation latency drift, instrumentation gaps in the recommendation pipeline, and early guardrail warnings on return rate. All subject to the same provisional-only constraint.

R. Redpanda/Kafka Upgrade Path

The vNext extension uses Python asyncio for the portfolio implementation. In a production deployment, the event stream would be backed by Redpanda or Kafka. Application logic remains broker-agnostic through the EventSource interface.

Question	Answer
Why not Kafka now?	Little marginal DS/Product Scientist signal. Risk of infrastructure cosplay. Reconciliation logic and provisional/authoritative distinction matter more than broker mechanics. asyncio delivers the same conceptual demonstration.
When does Redpanda become worth it?	When targeting data/platform engineering roles. When broker-specific interview prep is needed (offsets, partitions, consumer groups, exactly-once semantics). When demonstrating replay from Kafka offset rather than in-memory queue.
How does the swap work?	Replace AsyncSimulatorEventSource with RedpandaKafkaEventSource. All monitors and artifact writers are unchanged. EventSource interface guarantees broker-agnostic application logic.

S. Must-Have / Should-Have / Optional

Tier	Items
Must-Have (no credibility without)	Python async event simulator · Event schema with all required fields · SRMDetector with provisional_status · InstrumentationHealthMonitor · streaming_metric_snapshots (provisional_label=True) · consumer_lag_report · late_event_report · DLQ (dlq_events.parquet) · stream_batch_reconciliation_report · show_streaming_demo.py
Should-Have (high signal)	replay_run_log · provisional_anomaly_alerts with lag suppression · traffic_allocation_snapshots · duplicate_event_report · streaming_health_report · stream_quality_checks
Optional (stretch)	Redpanda/Kafka broker · Frontend dashboard · Cloud deployment · Flink/Spark Streaming · Online sequential testing · Real-time decision engine
Never Cut (non-negotiable)	Provisional vs authoritative distinction on every artifact · Stream-batch reconciliation · SRM early warning · Instrumentation health · Late-event handling · DLQ · No streaming-based final decisions anywhere in the system

T. Demo Narrative

30-Second Pitch

"MetaSignal's streaming extension monitors experiment health in the first hours, before the daily batch readout. Streaming detects broken assignment, platform event drops, and consumer lag in near real-time. What streaming cannot do is make final experiment decisions — that belongs to batch. The stream-batch reconciliation layer proves whether the provisional streaming alerts were right. Streaming gives speed. Batch gives authority. Reconciliation gives trust."

2-Minute Architecture Explanation

The async event simulator generates experiment events through an EventSource interface — AsyncSimulatorEventSource today, Redpanda-backed tomorrow if needed. Events flow through StreamValidator: bad events go to the DLQ, late events go to the late event report, duplicates are flagged. Valid events reach the monitors: SRMDetector on a 30-minute sliding window, InstrumentationHealthMonitor per platform/segment, ConsumerLagMonitor tracking p50/p95/p99 lag. RealtimeMetricSnapshotter aggregates provisional metrics every 5 minutes, always labelled provisional_label=True. After the daily batch run, StreamBatchReconciler compares provisional values to authoritative CUPED-adjusted batch values and writes stream_batch_reconciliation_report. If streaming agrees with batch consistently, the streaming layer is calibrated and trustworthy. If it diverges repeatedly, the log says so.

5-Minute Demo Walkthrough

- **Step 1.** python show_streaming_demo.py — starts the async event simulator at 100 events/second.
- **Step 2.** Inject SRM: simulator shifts to 54/46 split. After 500 users per variant, SRMDetector fires srm_streaming_alerts with deviation_pp=4.0, alert_level=warn.
- **Step 3.** Show srm_streaming_alerts artifact. provisional_status=pending_batch_confirmation. The alert is real. The decision is not.
- **Step 4.** Inject iOS drop: simulator stops emitting iOS events. InstrumentationHealthMonitor fires instrumentation_alerts within 2 windows. alert_level=critical, platform=ios.
- **Step 5.** Inject 20% late events. late_event_report.json updates. Provisional metric visibly undercounts.
- **Step 6.** Inject consumer lag spike. ConsumerLagMonitor: p95 > threshold. ProvisionalAnomalyDetector: lag_suppressed=True. No false anomaly alert fires.
- **Step 7.** Run daily batch compute. Official CUPED-adjusted values computed.

- **Step 8.** StreamBatchReconciler runs. Generates stream_batch_reconciliation_report.json. SRM alert: batch_confirmation_status=confirmed. iOS alert: confirmed. Late-event delta: minor_delta, suspected_cause=late_events.
- **Step 9.** Show streaming_health_report.json. Full system health summary.
- **Closing.** Streaming detected three real problems before the daily run. Batch confirmed two of them and explained the third. Reconciliation proved the streaming layer was working correctly.

U. Honest Resume and Interview Positioning

One-line claim

"MetaSignal Streaming Extension is a solo-built, non-production, production-simulated vNext extension that adds provisional SRM early warning and instrumentation health monitoring to MetaSignal. Streaming is provisional. Batch is authoritative. Reconciliation proves calibration."

What can be claimed

- Streaming architecture is designed broker-agnostic through an EventSource interface.
- SRM early warning is provisional, sample-gated, and lag-aware.
- Stream-batch reconciliation runs after every daily batch and documents deltas.
- Every streaming artifact carries provisional_label=True. No streaming artifact drives decisions.
- Failure scenarios include false alarms, batch suppression, and lag-aware suppression.
- The demo shows a provisional alert confirmed by batch, a provisional alert suppressed by batch, and a reconciliation with documented delta.

What must not be claimed

- "Real-time experiment decisioning" — streaming makes no decisions.
- "Streaming CUPED" — CUPED is a batch-only computation.
- "Kafka production pipeline" — this uses asyncio.
- "Live traffic monitoring" — the event source is a simulator.
- "Final experiment results from streaming" — final results are always batch.

How to explain the batch/streaming split

"Batch is authoritative because it applies CUPED, correct grain logic, denominator governance, and right-censoring. Streaming is fast but deliberately shallow — it counts events in windows and monitors allocation and instrumentation. The two serve different purposes. I designed reconciliation to prove the streaming layer is calibrated, not to pretend the two are interchangeable."

V. Red-Team Review

Q: Is this overengineering?

No. The streaming layer has a narrow scope: SRM early warning, instrumentation health, event quality. It does not replace CUPED, does not make decisions, and does not require a broker. The Python asyncio implementation is 4-6 days of focused work after MetaSignal core is complete.

Q: Why not just wait for batch?

For many experiments, waiting for batch is fine. The streaming layer earns its value in two scenarios: early experiments where a broken assignment at hour one invalidates all subsequent data, and high-stakes platform launches where an instrumentation failure must be caught before it corrupts the entire experiment corpus.

Q: Why not Kafka?

Kafka adds broker mechanics without adding statistical insight. The EventSource interface makes Kafka a drop-in replacement whenever it is needed. For a DS/Product Scientist portfolio, the reconciliation logic and provisional/authoritative distinction are far more valuable signals.

Q: Why not final readouts from streaming?

Streaming does not apply CUPED. It does not right-censor. It does not handle delayed maturity. It does not enforce denominator governance. A streaming CVR is a count ratio. A batch CVR is a variance-reduced, grain-correct, guardrail-gated metric. They are not the same number.

Q: What happens when streaming and batch disagree?

The reconciliation report documents the delta, estimates the suspected cause (late events, DLQ volume, CUPED adjustment, missing events), and flags whether the delta is within calibrated thresholds. Systematic unexplained delta triggers a `streaming_quality_flag` that labels streaming alerts as less trustworthy.

Q: How do you control false positives in streaming SRM?

Three mechanisms: minimum sample gate (no alert below 500 users per variant), minimum window duration (15 minutes), and lag-aware suppression (consumer lag above threshold suppresses SRM alerts because lag can create artificial split imbalance).

Q: How do late events affect provisional metrics?

Late events within `allowed_lateness_minutes` are included in the provisional metric, which can move after initial computation. Late events beyond the lateness window are quarantined to `late_event_report` and handled by batch. This is why provisional metrics carry a warning: they may change.

Q: How do you prevent analysts from peeking?

The display layer must make `provisional_label` unavoidable. Every metric snapshot is labelled PROVISIONAL — NOT FOR DECISIONS. The streaming demo does not show statistical significance, confidence intervals, or p-values. Those live in the batch experiment readout only.

Q: How do you know the streaming layer is trustworthy?

You know it is trustworthy when `stream_batch_reconciliation_report` consistently shows match or minor_delta with documented suspected causes. You know it is not trustworthy when `investigate_status` accumulates without explanation. Reconciliation is the evidence.

Q: What happens if the streaming layer fails?

MetaSignal v1 continues operating normally. The streaming layer is optional and additive. A consumer failure triggers replay after recovery. Batch truth is never affected. `replay_run_log` documents the repair.

W. Recommendation Before Execution Planning

This extension should stay parked vNext. Do not implement it before MetaSignal core is working — meaning CUPED is validated, A/A suite passes, guardrail-first decisioning operates, and the 60-day simulated operational history is complete.

Minimum implementation when ready: 4-6 focused days. Day 1-2: stream_event_generator, EventSource interface, StreamValidator, DLQ. Day 3: SRMDetector, InstrumentationHealthMonitor. Day 4: RealtimeMetricSnapshotter, streaming_metric_snapshots. Day 5: StreamBatchReconciler. Day 6: show_streaming_demo.py with all 10 failure scenarios.

Do not build Kafka or Redpanda. asyncio delivers all the conceptual signal. If broker mechanics are needed later, the EventSource interface makes the swap clean.

The extension is additive depth, not a requirement. MetaSignal v1 with CUPED, A/A validation, guardrail-first decisioning, and a 60-day operational history is already a strong portfolio project. Streaming is the depth layer that makes a reader say: this person understands the full stack.

Streaming gives speed. Batch gives authority. Reconciliation gives trust.

MetaSignal Streaming Extension PRD vNext · April 2026 · Solo-built, non-production, production-simulated · Batch remains authoritative